

SPA JS: Building Cross-Platform SPAs with Less Code

SPA JS is a single page application that is simplified and developed using JavaScript. It is used to create a cross-platform application using which you can develop a Web app, and the same code can also be used to develop desktop apps. If the code is bundled with the Electron software, then the result would be a desktop app, and if you put the code under a Cordova stack, then you would get Android or iOS applications. So, you can use the same code of SPA JS to develop applications for the desktop as well as the mobile.



The primary aim of SPA JS is to keep development simple and eradicate repetitions. It also follows the crucial principle, YAGNI (You Ain't Gonna Need It), which states that you should "Always implement things only when you need them but never when you just think that you *may* need them." For example, in open source software, contributors may develop different modules by assuming that you *may need them*. However, these modules may not be useful at all.

Why SPA JS?

Many of you may be familiar with Angular, React and the Vue.js framework but very few may be aware

about frameworks like Knockout, Ember and Backbone. It's not that Angular, React, etc, are really game changers. We already had the concept of building a single page application using these frameworks. So, let us examine the problem with Angular, React or Vue.js, as well as other programming options.

Problem 1: The question is why do we have different frameworks? This is because developers try to build some applications on one of these frameworks, but the latter do not solve their particular problem in the way they want to. So, developers try to create their own frameworks that solve their individual problem, but this leads to a lot of frameworks.

Problem 2: Earlier, we had frontend designers and backend developers. The former designed the page, while the latter used programming languages like C, C++, Java and .NET to build a page and call APIs. JavaScript was then introduced as frontends for micro-interactions, but frontend designers faced difficulties designing pages with it. To address this situation, backend developers were brought to the frontend. But unfortunately, backend developers are generally not good in design.

So, companies and developers are facing the issue of how to create a perfect bridge between the frontend and backend. Recently, the term 'full-stack developer' has emerged, describing those who know all the backend development,

